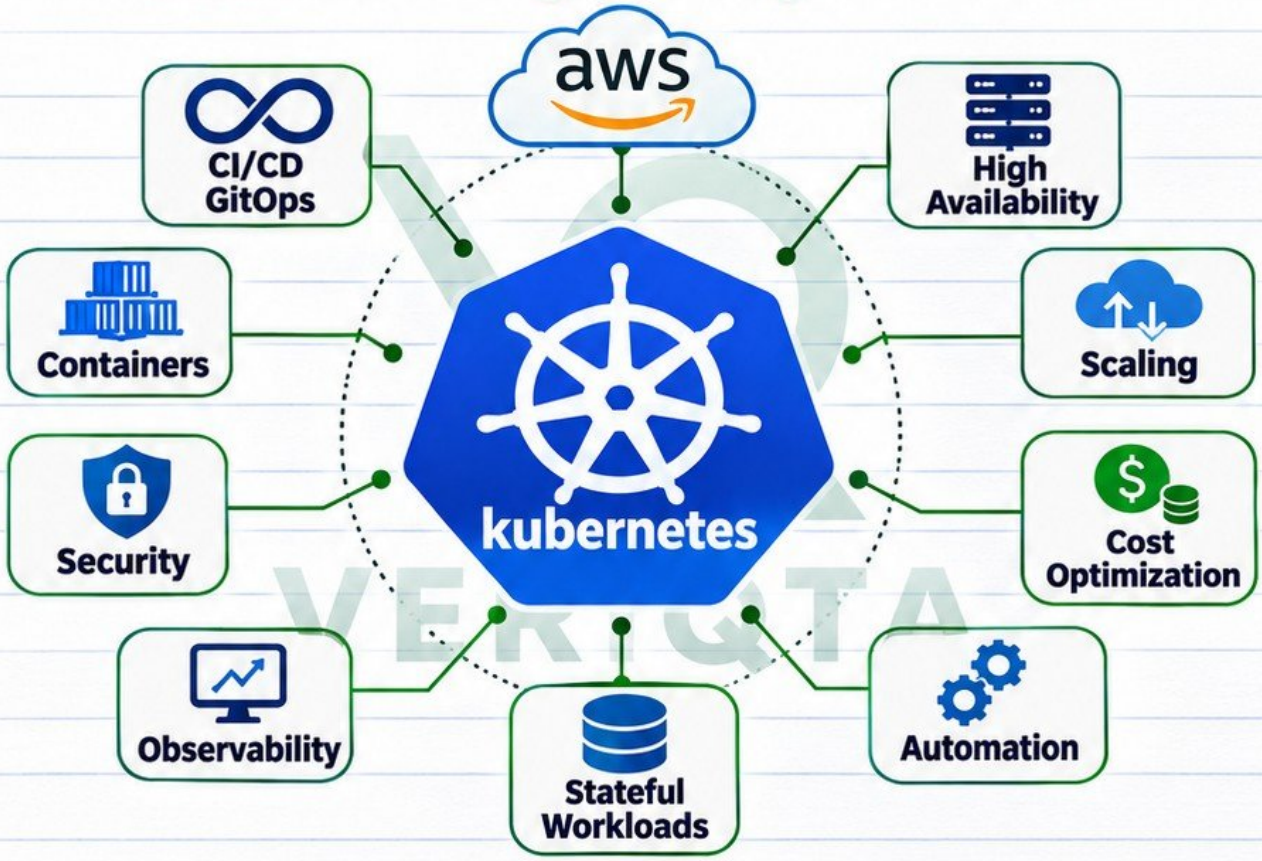
VERIQTA

20 Kubernetes Production Projects

Real-World Architecture, Failures, Recovery and Senior Engineering Decisions



- Reliability Engineering
- Security and Zero Trust
- Advanced Operations
- Disaster Recovery (DR)
- Platform Engineering (IDP)
- Cost and Performance

@veriqta Instagram | @veriqta GitHub | @veriqta LinkedIn | @veriqta Telegram | @veriqta X | @veriqta Discord

VERIQTA

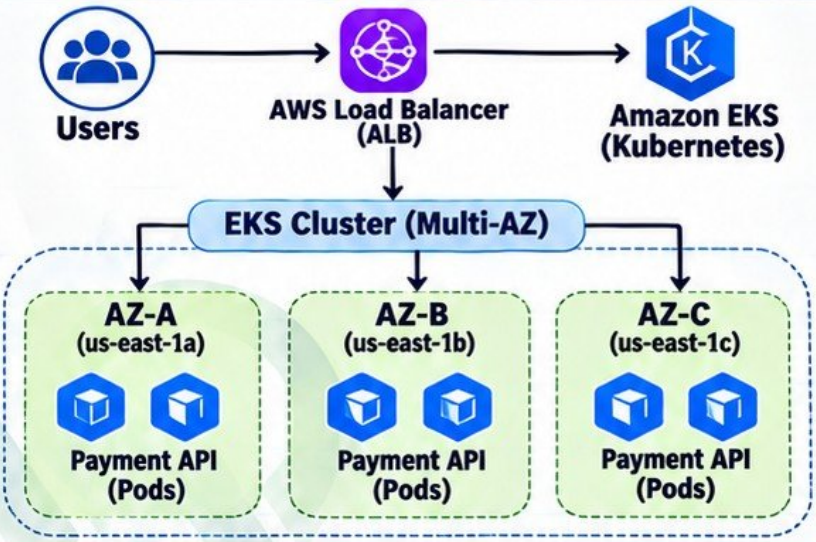
The Kubernetes Cluster That Survived an Availability Zone Failure

Project 01: Build an AZ-resilient payment authorization platform.

Project Objectives

- Deploy payment API replicas across three AWS Availability Zones.
- Configure topology spread constraints, pod anti-affinity, PodDisruptionBudgets and node provisioning.
- Simulate an AZ failure and investigate endpoint removal, pod rescheduling and load balancer routing.
- Determine whether the remaining zones have sufficient capacity without triggering cascading failures.

Architecture Overview



Key Kubernetes Configurations

- **Topology Spread Constraints** (Balance pods across AZs)
- **Pod Anti-Affinity** (Avoid pods on same node)
- **PodDisruptionBudget (PDB)** (Maintain minimum replicas)
- **Node Provisioning** (Multi-AZ capacity and autoscaling)

Production Challenge

The cluster remains healthy, but payment requests fail because the surviving zones cannot absorb the workload.

Evidence to Collect

- Request success rate (e.g., ≥ 99.9%).
- p99 latency (before, during, after AZ failure).
- Available replicas and pod distribution.
- Capacity headroom (CPU, memory, and nodes).
- Recovery timeline (time to stable performance).

Expected Outcome

- Service continues with acceptable performance.
- Pods reschedule automatically to healthy AZs.
- Load balancer routes traffic only to available endpoints.
- No cascading failures on dependent services.

Skills Demonstrated

- High-availability Kubernetes design.
- Workload resilience and failure testing.
- Production capacity planning and vaiidation.

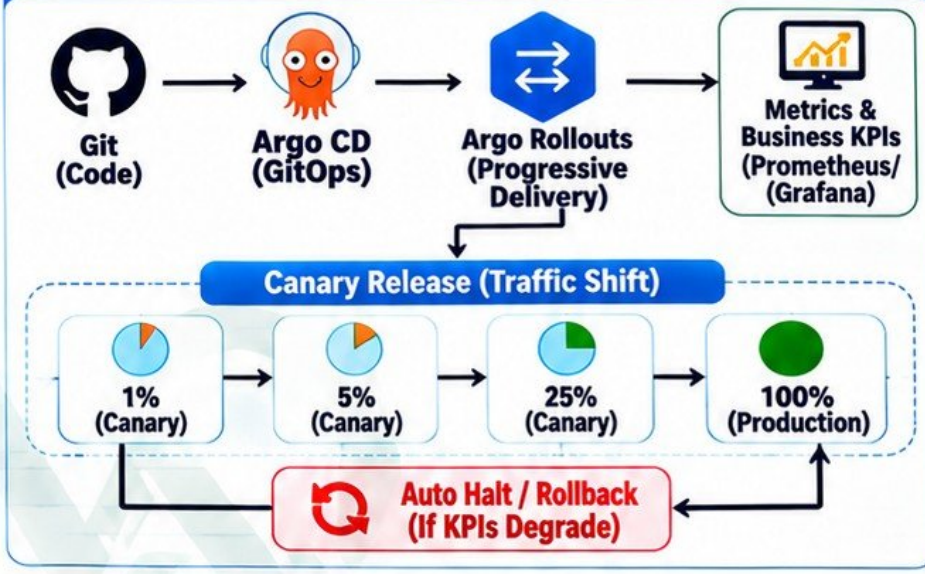
The Deployment That Passed CI/CD but Caused a Revenue Outage

Project 02: Engineer a risk-aware progressive delivery system.

Project Objectives

- Build Argo CD GitOps deployment workflows with Argo Rollouts.
- Release a new checkout service to 1%, 5%, 25% and 100% of traffic.
- Evaluate error budgets, latency, conversion failures and business transaction metrics.
- Automatically halt promotion and roll back when canary health deteriorates.

Architecture and Delivery Flow



Key Tools and Components

- **Kubernetes** (Workload platform)
- **Argo CD** (GitOps for deployments)
- **Argo Rollouts** (Canary and progressive delivery)
- **Prometheus** (Metrics and SLOs)
- **Grafana** (Visualization and alerting)

Production Challenge

HTTP responses remain successful, but checkout completion rates collapse, leading to revenue loss.

Evidence to Collect

- Canary analysis (traffic, errors, and KPIs).
- Deployment history (promotions and rollback).
- Transaction metrics (checkout success rate, conversion rate, and revenue impact).
- Rollback time (from detection to full recovery).

Expected Outcome

- Safe and controlled production releases.
- Automatic halt and rollback on KPI degradation.
- Protected revenue and customer experience.
- Clear deployment and rollback audit trail.

Skills Demonstrated

- GitOps and progressive delivery design.
- SLO/error budget and business KPI monitoring.
- Production incident prevention and recovery.

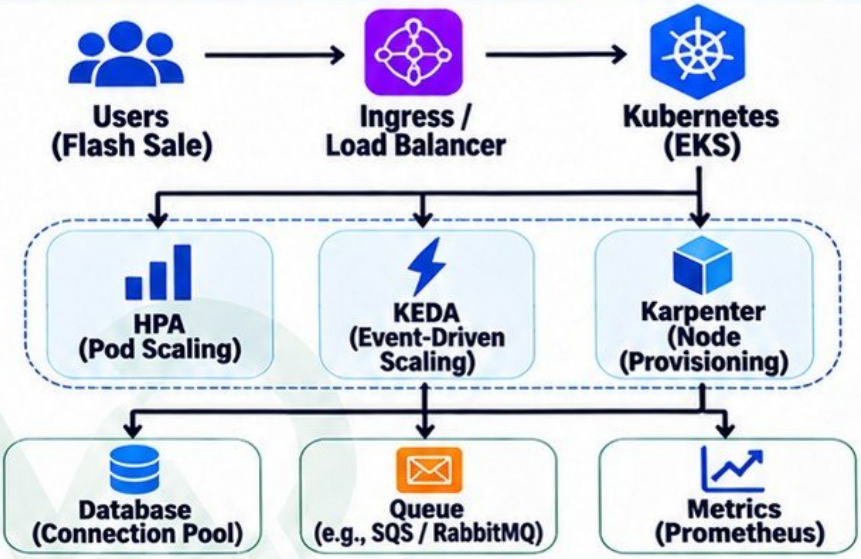
The Autoscaler That Made a Traffic Spike Worse

Project 03: Build a workload-aware scaling system that prevents retry storms.

Project Objectives

- Implement HPA, KEDA and node autoscaling with Karpenter.
- Simulate a flash sale with sudden traffic and queue growth.
- Investigate CPU saturation, cold starts, insufficient nodes, database connection exhaustion and retries.
- Compare scaling on CPU, request concurrency, queue age and application-specific metrics.

Architecture and Scaling Flow



Key Components

- **HPA** (Scale pods on metrics)
- **KEDA** (Scale on events, e.g., queue lag)
- **Karpenter** (Provision right-sized nodes)
- **Connection Pooling** (Limit and manage DB connections)
- **Autoscaling Metrics** (CPU, concurrency, queue age, custom)

Production Challenge

More replicas create more database connections and increase failure rates during a traffic spike.

Evidence to Collect

- Scaling timelines (pods and nodes).
- Queue latency (before, during, after spike).
- Database connection utilization and limits.
- p99 latency (user request end-to-end).
- Dropped or failed requests (error rate).

Expected Outcome

- Workloads scale based on real demand.
- No retry storms or database exhaustion.
- Stable latency and high success rate.
- Right-sized, cost-efficient node scaling.

Skills Demonstrated

- Workload-aware autoscaling design.
- Traffic spike and capacity testing.
- Performance and failure analysis.
- Production optimization and cost control.

VERIQTA

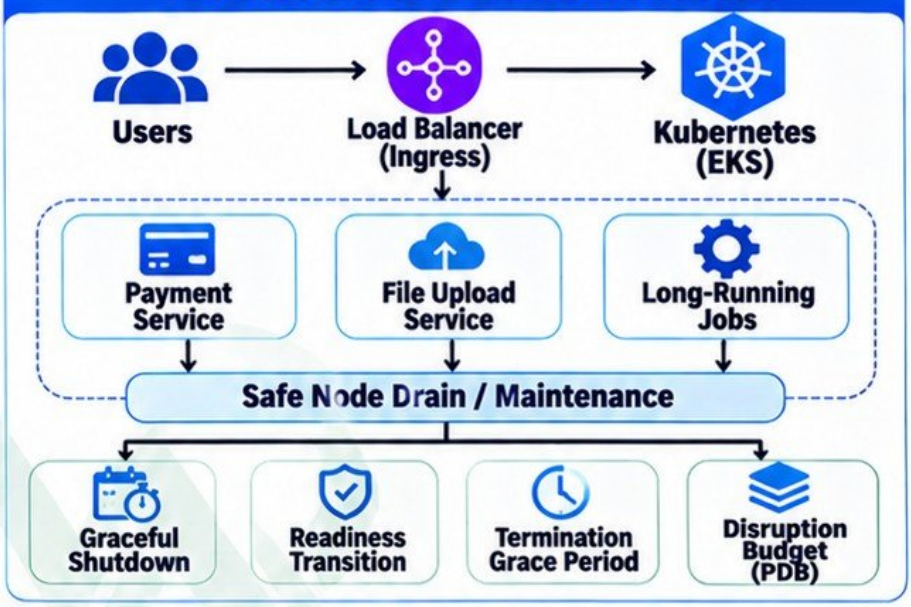
The Node Drain That Terminated Thousands of Active Transactions

Project 04: Engineer safe Kubernetes maintenance for long-running workloads.

Project Objectives

- Build a service that handles payments, file uploads and long-running requests.
- Configure graceful shutdown, readiness transitions, termination grace periods and disruption budgets.
- Simulate managed node upgrades, voluntary evictions and abrupt instance loss.
- Preserve in-flight operations using idempotency keys and durable work queues.

Architecture and Maintenance Flow



Key Kubernetes Configurations

- **Graceful Shutdown** (preStop hook and SIGTERM)
- **Readiness Transitions** (traffic removal before termination)
- **Termination Grace Period** (complete in-flight requests)
- **PodDisruptionBudget (PDB)** (maintain minimum available pods)
- **Durable Work Queues** (SQS / RabbitMQ / Kafka)
- **Idempotency Keys** (prevent duplicate processing)

Production Challenge

A routine node replacement interrupts financial transactions and causes duplicate processing.

Evidence to Collect

- Completed transaction count and success rate.
- Interrupted or failed requests.
- Duplicate transaction count (before and after fix).
- Maintenance duration (drain, upgrade, recovery).

Expected Outcome

- No transaction loss during node maintenance.
- In-flight requests complete successfully.
- Zero or minimal duplicate processing.
- Controlled, predictable node upgrades.

Skills Demonstrated

- Production-grade workload lifecycle design.
- Reliability and data integrity in Kubernetes.
- Safe maintenance and upgrade operations.
- Resilient handling of long-running requests.

VERIQTA

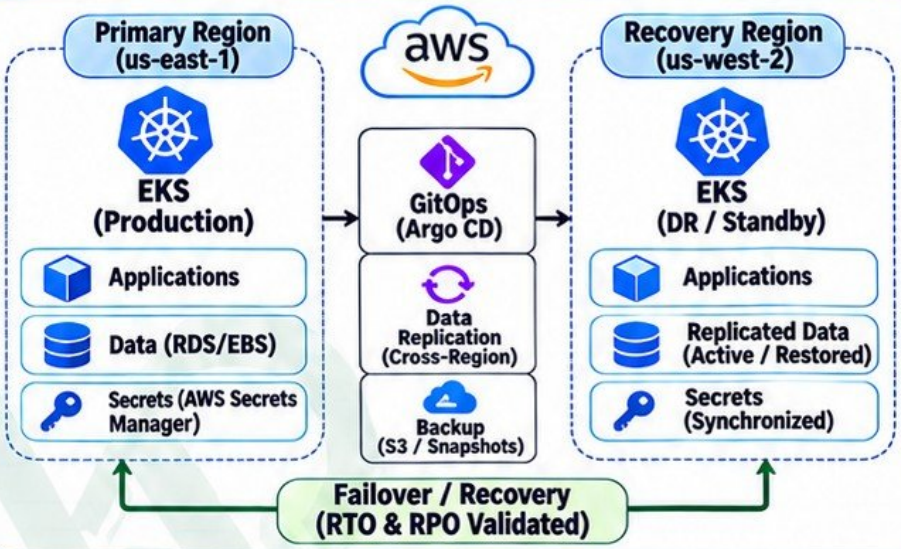
The Disaster Recovery Cluster That Could Not Restore the Business

Project 05: Build and validate a regional Kubernetes disaster recovery platform.

Project Objectives

- Provision primary and recovery EKS clusters in separate AWS Regions.
- Reconstruct infrastructure, GitOps applications, secrets and external dependencies.
- Restore persistent data and verify application-level consistency.
- Conduct a timed regional outage simulation against defined RTO and RPO targets.

Multi-Region DR Architecture (AWS EKS)



Key Components

- **Amazon EKS** (Primary and DR clusters)
- **Argo CD** (GitOps and environment sync)
- **Data Replication** (RDS/EBS, cross-region backup)
- **AWS Secrets Manager** (Secrets and configuration)
- **S3 / Snapshots** (Backups and recovery)
- **Route 53 / Global Accelerator** (Traffic failover and routing)

Production Challenge

Applications restart in the recovery region, but critical data or authentication dependencies are unavailable.

Evidence to Collect

- Actual RTO (time to restore service).
- Measured RPO (maximum data loss).
- Data integrity results (consistency and validation).
- Recovery runbook (steps, owners and outcomes).

Expected Outcome

- Successful regional failover and restore.
- RTO and RPO meet defined business targets.
- No data loss or temporary data inconsistency.
- Documented, repeatable recovery process.

Skills Demonstrated

- Multi-region Kubernetes architecture.
- Disaster recovery and business continuity.
- Data protection, replication and validation.
- Incident simulation and recovery execution.

VERIQTA

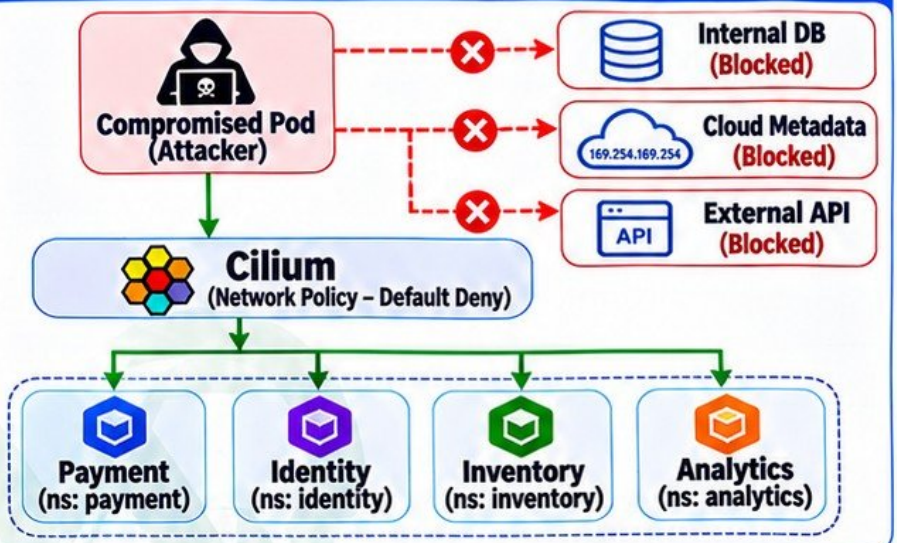
The Compromised Pod That Tried to Reach Every Service

Project 06: Build a zero-trust Kubernetes network with attack-path verification.

Project Objectives

- Deploy separate payment, identity, inventory and analytics workloads.
- Enforce default-deny ingress and egress with Cilium network policies.
- Restrict DNS, external API access and cross-namespace communication.
- Simulate a compromised application attempting unauthorized lateral movement.

Architecture and Security Flow



Key Components

- **Kubernetes (EKS)** (Cluster and namespaces)
- **Cilium** (eBPF networking and policies)
- **Network Policies** (Default deny, allow-lists)
- **DNS Controls** (Restrict and audit DNS access)
- **Egress Controls** (Limit external API and metadata access)
- **Workload Identity** (Service accounts and least privilege)

Production Challenge

An attacker gains access to one pod and attempts to reach internal databases and cloud metadata services.

Evidence to Collect

- Allowed and denied network flows (Cilium Hubble).
- Network-policy verification tests (connectivity matrix).
- Identity boundaries (service accounts and access scope).
- Attack-path analysis (before and after policy).

Expected Outcome

- Default-deny network with controlled access.
- Blocked lateral movement to sensitive services.
- Restricted DNS and external access.
- Verified and documented attack-path results.

Skills Demonstrated

- Zero-trust Kubernetes networking.
- Cilium policy design and validation.
- Threat simulation and attack-path analysis.
- Production-grade security hardening.

VERIQTA

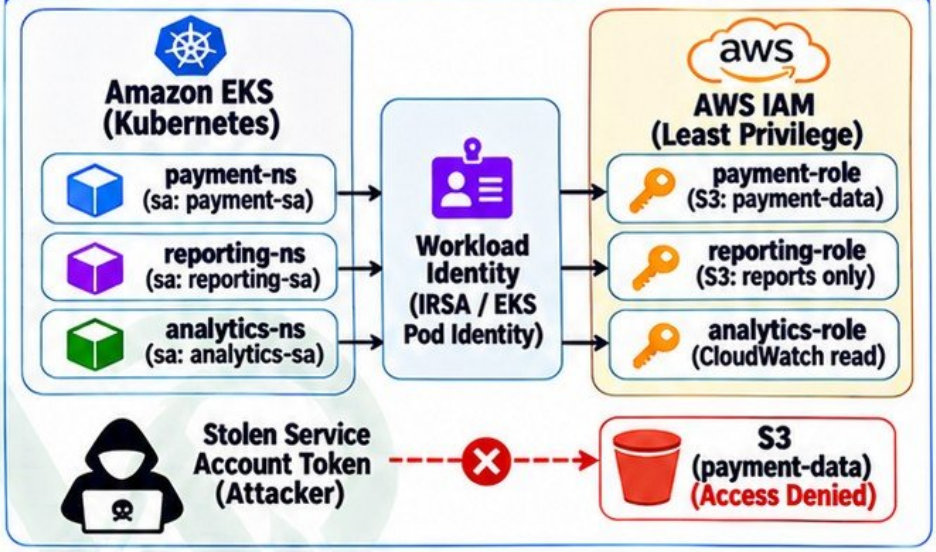
The Kubernetes Service Account That Exposed the AWS Account

Project 07: Engineer workload identity and cloud permission isolation.

Project Objectives

- Configure EKS Pod Identity or IRSA with narrowly scoped IAM roles.
- Separate cloud permissions by workload, namespace and environment.
- Prevent applications from inheriting unnecessary node-level permissions.
- Simulate stolen service account tokens and attempted cross-service AWS access.

Architecture and Access Flow



Key Components

- **Amazon EKS** (Managed Kubernetes platform)
- **EKS Pod Identity / IRSA** (Workload identity to IAM)
- **AWS IAM Roles & Policies** (Least-privilege access)
- **Amazon S3** (Payment and reporting data)
- **AWS CloudTrail** (API activity and audit logs)
- **Service Accounts & Namespaces** (Identity and isolation)

Production Challenge

A compromised reporting application attempts to access production payment data stored in S3.

Evidence to Collect

- IAM authorization tests (allowed vs denied).
- AWS CloudTrail events (access attempts).
- Denied API calls (S3 and other services).
- Permission review (roles, policies, and scope).

Expected Outcome

- Workloads access only approved AWS resources.
- Cross-service and cross-namespace access denied.
- Unauthorized S3 access attempts are logged and blocked.
- Clear audit trail and permission boundaries.

Skills Demonstrated

- Kubernetes workload identity (IRSA/EKS).
- IAM policy design and least privilege.
- Security testing and access validation.
- Audit, detection and incident investigation.

VERIQTA

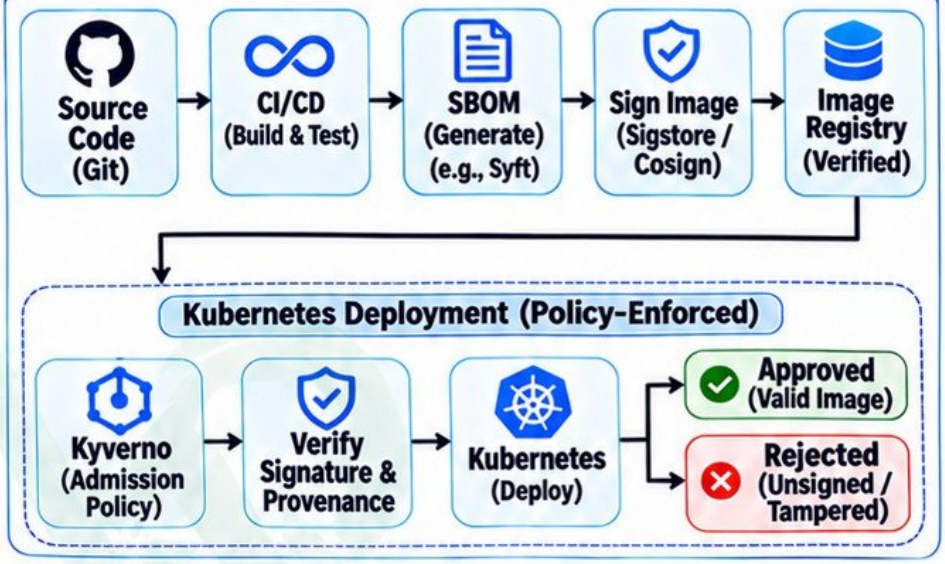
The Container Image That Passed Every Deployment Check

Project 08: Build a verifiable software supply chain for Kubernetes.

Project Objectives

- Generate SBOMs and scan images for vulnerabilities.
- Sign container images and verify provenance using Sigstore tooling.
- Enforce deployment admission rules with Kyverno or an equivalent policy engine.
- Simulate an unsigned image, altered artifact and compromised build workflow.

Secure Software Supply Chain Flow



Key Components

- **Container Image** (Build, tag and push)
- **SBOM (Software Bill of Materials)** (Track all dependencies)
- **Sigstore / Cosign** (Sign and verify image provenance)
- **Kyverno** (Policy-based admission control)
- **Container Registry** (Store signed and verified images)
- **Vulnerability Scanning** (Trivy / Grype / ECR scan)

Production Challenge

An attacker replaces a trusted application image after the CI pipeline completes.

Evidence to Collect

- Signature verification results (e.g., cosign verify).
- Provenance and build attestation (SLSA).
- Admission logs (Kyverno allow / deny).
- Incident investigation trail (who, what, when).

Expected Outcome

- Only signed and trusted images are deployed.
- Tampered or unsigned images are rejected.
- Full traceability from code to production.
- Reduced supply chain and deployment risk.

Skills Demonstrated

- Kubernetes supply chain security.
- Image signing and provenance (Sigstore).
- Policy enforcement with Kyverno.
- Security incident investigation and response.

VERIQTA

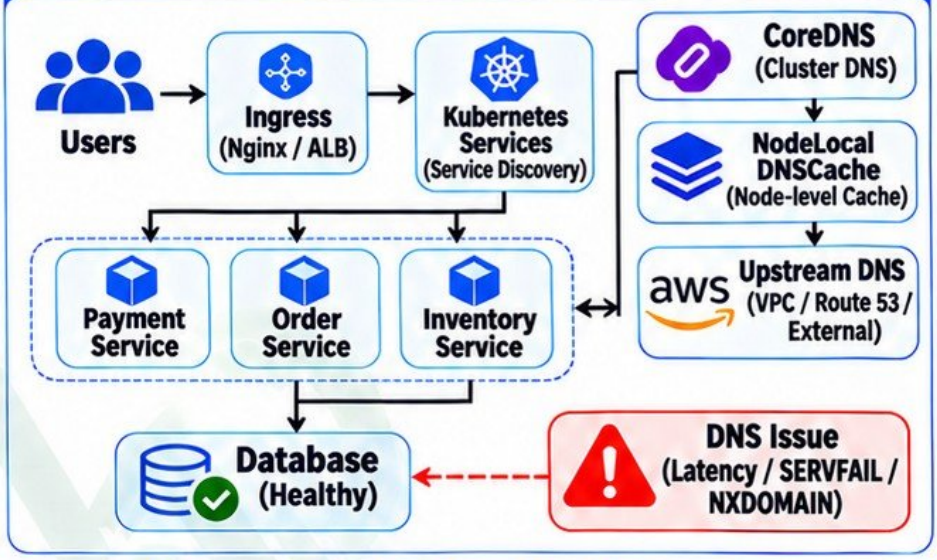
The DNS Outage That Looked Like a Database Failure

Project 09: Build a Kubernetes DNS resilience and forensic investigation lab.

Project Objectives

- Deploy microservices with high-volume service discovery requirements.
- Investigate CoreDNS capacity, DNS caching, upstream resolution and NodeLocal DNSCache.
- Introduce DNS latency, resolver failures and incorrect records.
- Correlate DNS failures with connection errors, application traces and downstream timeouts.

Architecture and DNS Investigation Flow



Key Components

- **Kubernetes Services** (Service discovery, ClusterIP)
- **CoreDNS** (Cluster DNS and plugins)
- **NodeLocal DNSCache** (Node-level DNS caching)
- **Upstream DNS** (VPC Resolver / Route 53)
- **Monitoring & Tracing** (Prometheus, Grafana, OpenTelemetry)
- **DNS Testing Tools** (dig, nslookup, kdig, kubectl)

Production Challenge

Applications report database failures even though the database is healthy.

Evidence to Collect

- DNS query latency and error rates.
- SERVFAIL / NXDOMAIN counts.
- CoreDNS resource usage and saturation.
- Application recovery timeline after DNS fix.

Expected Outcome

- Identify DNS as the root cause.
- Validate CoreDNS and cache configuration.
- Restore normal service discovery and traffic.
- Prevent repeat DNS-related outages.

Skills Demonstrated

- Kubernetes DNS architecture and tuning.
- CoreDNS and NodeLocal DNSCache operations.
- Troubleshooting and DNS forensics.
- Production DNS monitoring and resilience.

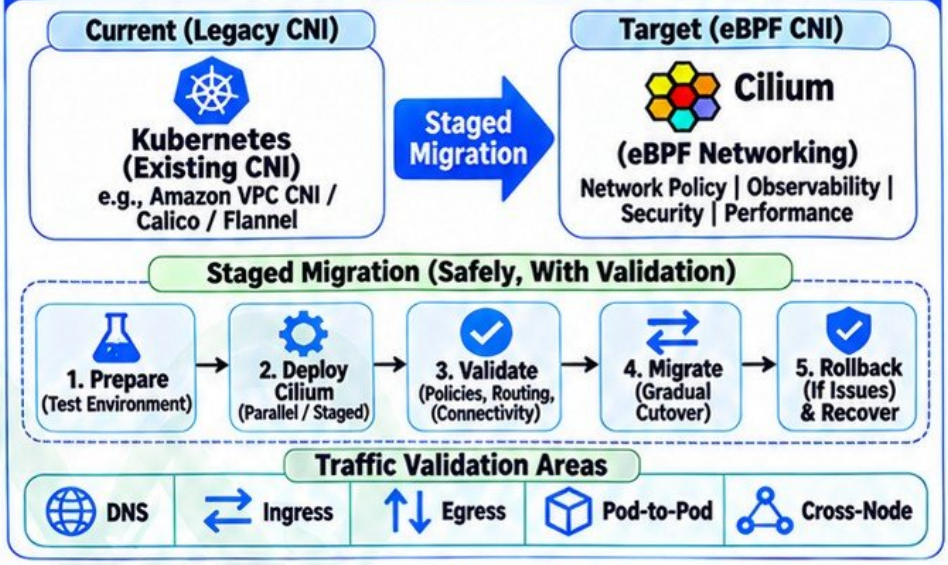
The Network Policy Migration That Silently Broke Production

Project 10: Migrate a production Kubernetes network to eBPF-based networking.

Project Objectives

- Design a staged CNI migration strategy in a disposable, production-like environment.
- Validate Cilium policies, service routing, observability and node-level connectivity.
- Test DNS, ingress, egress, pod-to-pod traffic and cross-node communication.
- Create rollback and recovery procedures for partial migration failure.

CNI Migration Strategy and Traffic Flow



Key Components

- **Kubernetes Cluster** (Production-like environment)
- **Cilium (eBPF CNI)** (Networking, policies and observability)
- **Network Policies** (Ingress, egress and service-to-service)
- **Observability** (Hubble, Prometheus, Grafana)
- **Migration & Rollback** (Staged cutover and recovery)
- **Validation Tools** (kubectl, cilium, Hubble, conn tests)

Production Challenge

Network policy behavior changes during migration and breaks traffic between services that previously communicated.

Evidence to Collect

- Connectivity matrix (before vs after migration).
- Packet-flow analysis (Hubble / eBPF traces).
- Network policy audit (rules, changes and impact).
- Recovery results (rollback time and service health).

Expected Outcome

- Successful, low-risk CNI migration.
- All intended traffic paths validated.
- No unintended service-to-service blocks.
- Documented rollback and recovery process.

Skills Demonstrated

- Kubernetes networking and CNI migration.
- Cilium (eBPF) policy design and validation.
- Traffic analysis and production troubleshooting.
- Operational change management and rollback.

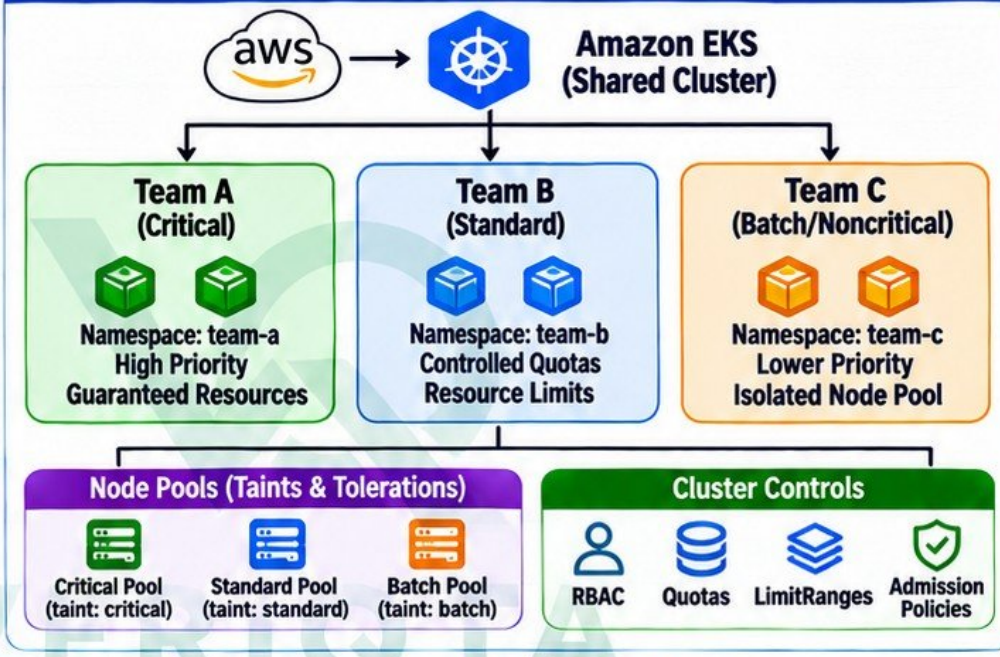
The Multi-Tenant Cluster Where One Team Starved Everyone Else

Project 11 Build a production multi-tenant Kubernetes platform with enforced isolation.

Project Objectives

- Onboard multiple engineering teams into a shared EKS cluster.
- Configure RBAC, resource quotas, LimitRanges and admission policies.
- Separate critical and noncritical workloads using node pools, taints and tolerations.
- Simulate a tenant consuming excessive CPU, memory, storage and Kubernetes API requests.

Multi-Tenant EKS Architecture



Production Challenge

A poorly configured batch workload prevents another team's critical service from scaling.

Implementation Steps

1. Create dedicated namespaces for each team.
2. Apply RBAC for least-privilege access.
3. Define ResourceQuota and LimitRange per namespace.
4. Enforce policies with an admission controller (e.g., Kyverno/Gatekeeper).
5. Configure node pools and assign taints/tolerations.
6. Run a load test to simulate a resource-hungry tenant.

Evidence and Validation

- Tenant isolation tests (no cross-impact).
- Resource allocation and quota enforcement.
- Kubernetes API responsiveness under load.
- Service SLOs for critical workloads (e.g., 99.9% availability, p95 latency).

Key Takeaways

- Enforce isolation and fair resource usage.
- Protect critical workloads with priority and dedicated capacity.
- Use quotas, policies and node pools to prevent noisy neighbors.
- Continuously test platform limits and SLOs.

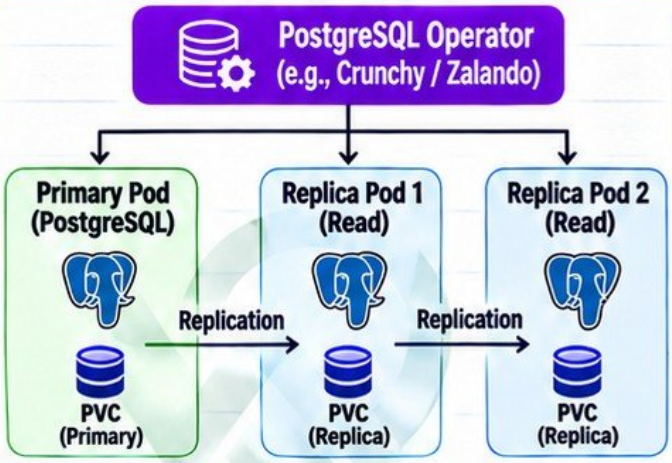
The Persistent Volume That Survived the Pod but Lost the Data

Project 12 Engineer stateful Kubernetes recovery with verified data integrity.

Project Objectives

- Deploy a production-like PostgreSQL workload using a Kubernetes operator.
- Implement persistent storage, backups, point-in-time recovery (PITR) and replication.
- Simulate node loss, volume unavailability, storage exhaustion and database corruption.
- Compare Kubernetes volume recovery with application-consistent database restoration.

Stateful PostgreSQL on Kubernetes



Data Protection and Recovery

- Automated Backups (pg_dump / pgBackRest)
- Point-in-Time Recovery (PITR)
- Replication (High Availability)
- Volume Snapshots (EBS / CSI)
- Data Integrity Checks (Consistency, Hash, Count)

Infrastructure and Storage

EKS / Kubernetes · EBS / CSI (Persistent Storage) · S3 (Backups) · KMS (Encryption) · Monitoring & Alerts

Production Challenge

Kubernetes reports healthy pods, but the database contains missing or inconsistent transactions.

Failure Scenarios to Test

- Node Loss (Pod Reschedule)
- Volume Unavailable (EBS/CSI)
- Storage Exhaustion (Full Disk)
- Database Corruption (Data Inconsistency)

Evidence and Validation

- Recovery point (RPO) and recovery time (RTO).
- Restored transaction count matches source.
- Data integrity checks (e.g., checksums, row counts).
- Replication health and consistency status.
- Comparison of PVC recovery vs database PITR restoration.

Key Takeaways

- A healthy pod does not guarantee data correctness.
- Use an operator for automated backups, PITR and replication.
- Validate data integrity after every recovery.
- Test failure and corruption scenarios regularly.
- Prefer application-consistent backups over volume-only recovery for stateful data.

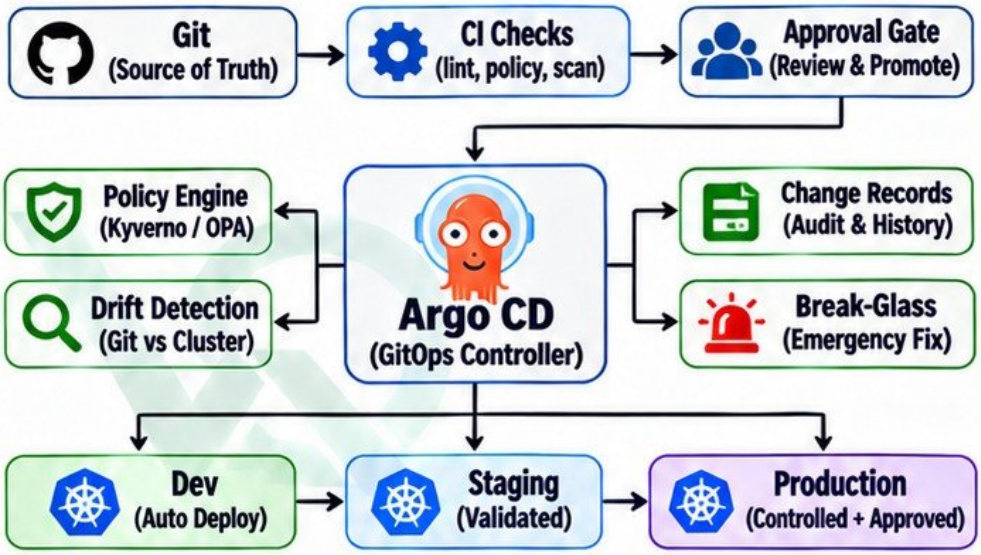
The GitOps Platform That Repeatedly Reintroduced a Security Vulnerability

Project 13 Build a policy-controlled GitOps platform with emergency change governance.

Project Objectives

- Deploy Argo CD with environment-specific repositories and controlled promotion.
- Detect drift between Git and live Kubernetes resources.
- Introduce approval gates, policy checks, change records and break-glass procedures.
- Test how to reconcile an emergency security fix without reverting it accidentally.

GitOps Architecture with Governance



Production Challenge

An engineer patches a vulnerability during an incident, but automated reconciliation restores the vulnerable configuration from Git.

Implementation Flow

1. Set up Argo CD and environment-specific repositories.
2. Enforce policy checks (e.g., Kyverno/OPA) in the deployment pipeline.
3. Enable drift detection and alerting.
4. Define approval gates for promotion to production.
5. Implement a break-glass process for emergency fixes.
6. Merge the emergency fix to Git and validate reconciliation behavior.

Evidence and Validation

- Git history (commit, author, time, and change).
- Argo CD audit logs and reconciliation events.
- Policy decision results (allow, deny, or override).
- Drift detection before and after the emergency fix.
- Incident timeline (detection, mitigation, permanent fix).

Key Takeaways

- Git must remain the source of truth, even for emergency fixes.
- Use approval gates and policy checks to prevent unsafe changes.
- Enable drift detection and change auditing.
- Have a documented break-glass process.
- Validate that the fix stays applied after reconciliation.

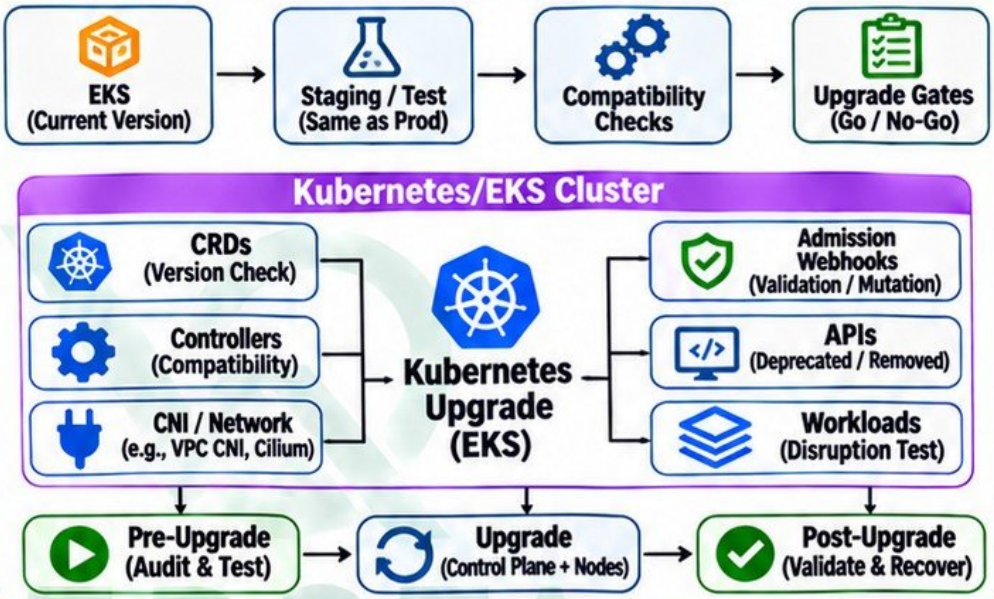
The Kubernetes Upgrade That Broke a Critical Admission Webhook

Project 14 Build a zero-surprise Kubernetes upgrade qualification system.

Project Objectives

- Create a representative staging environment for EKS version upgrades.
- Audit deprecated APIs, CRDs, controllers, admission webhooks and CNI compatibility.
- Test node replacement, workload disruption and supporting controller failures.
- Establish upgrade gates and a realistic recovery plan for incompatible components.

Upgrade Qualification Workflow



Production Challenge

The control plane upgrades successfully, but a broken admission webhook blocks new workloads (e.g., pods, deployments, namespace creation).

Key Implementation Steps

1. Mirror production in a staging EKS environment.
2. Run deprecation and API/CRD audit (e.g., kubent, pluto).
3. Validate controllers, webhooks and CNI compatibility.
4. Test node upgrades and workload disruption.
5. Define upgrade gates and rollback criteria.
6. Document and test a recovery plan (disable, fix or upgrade webhook).

Evidence and Validation

- Compatibility report (APIs, CRDs, controllers).
- Admission webhook health and validation tests.
- Deployment and workload creation after upgrade.
- Upgrade timeline and disruption metrics (e.g., pod restarts, downtime).
- Recovery decision and result (fix, disable or upgrade).

Key Takeaways

- Always test upgrades in a production-like staging environment.
- Include admission webhooks and controllers in upgrade validation.
- Use upgrade gates with clear go/no-go criteria.
- Have a documented and tested rollback or workaround.
- Keep a compatibility and deprecation inventory.

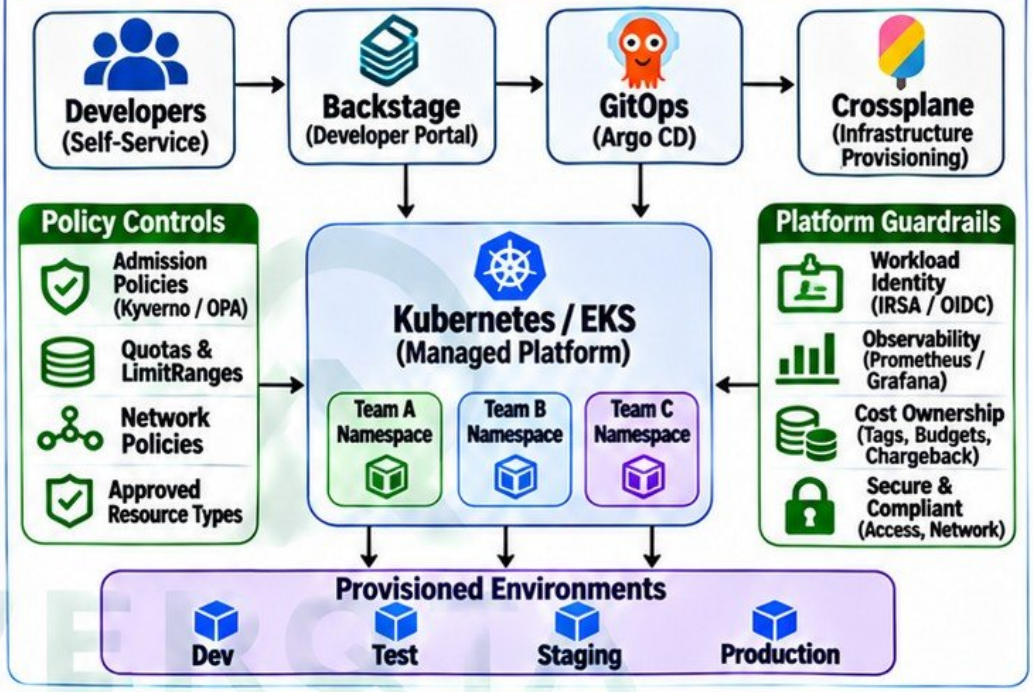
The Internal Developer Platform That Provisioned Unsafe Infrastructure

Project 15 Build a governed self-service Kubernetes application platform.

Project Objectives

- Create a developer portal using Backstage.
- Provision approved application environments using Crossplane and GitOps.
- Standardize workload identity, networking, observability, quotas and cost ownership.
- Implement policy controls that reject unsafe infrastructure requests.

Platform Architecture: Self-Service and Governed



Production Challenge

Developers can create environments quickly, but ungoverned self-service exposes databases and generates unexpected cloud costs.

Implementation Steps

1. Deploy Backstage and integrate with Git (e.g., GitHub).
2. Configure Crossplane to provision cloud resources (e.g., VPC, RDS, S3).
3. Set up Argo CD (GitOps) for application and infrastructure deployment.
4. Enforce policies (Kyverno/OPA), quotas, networking and tagging.
5. Implement observability and cost ownership dashboards.
6. Test requests (valid and invalid) and validate policy enforcement and teardown.

Evidence and Validation

- Provisioning time (e.g., < 10 minutes).
- Policy violations (blocked unsafe requests).
- Infrastructure inventory (approved resources).
- Ownership records (tags, team and cost).
- Teardown verification (resources removed, no orphaned costs).

Key Takeaways

- Enable fast, self-service delivery without sacrificing security or cost control.
- Use Backstage + Crossplane + GitOps for a scalable, governed platform.
- Enforce policy, quotas and networking as guardrails.
- Track ownership, usage and cost for every environment.
- Automate environment lifecycle, including teardown.

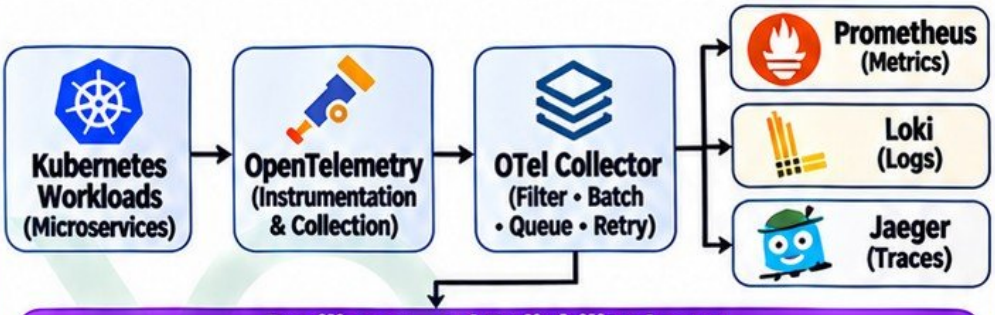
The Monitoring System That Failed During the Incident

Project 16 Build an observability platform that remains useful during partial outages.

Project Objectives

- Instrument applications using OpenTelemetry.
- Collect metrics, logs and traces with Prometheus-compatible monitoring and distributed tracing.
- Test cardinality explosions, telemetry backpressure, collector failures and missing traces.
- Design alerting that distinguishes customer impact from infrastructure noise.

Observability Architecture (Production-Grade)



Resilience and Reliability Layer

Buffering (Queue) | Backpressure Control | High Availability (HA Collectors) | Data Retention & Sampling | Degradation Mode (During Outage)

Alerting and Insight

SLO / SLI Alerts (Customer Impact) | Anomaly Detection (Real Issues) | Noise Reduction (Fewer False Alerts) | Incident Investigation (Metrics + Logs + Traces)

Production Challenge

A major incident generates so much telemetry that the monitoring platform becomes unavailable.

Implementation Steps

1. Instrument services with OpenTelemetry (traces, metrics, logs).
2. Deploy OTel Collectors with buffering, retries and rate limits.
3. Integrate Prometheus, Loki and Jaeger (or Grafana Tempo).
4. Simulate high-cardinality and collector failure scenarios.
5. Create SLO-based alerts to focus on customer impact.
6. Test system behavior under partial outage and recovery.

Evidence and Validation

- Telemetry loss rate (e.g., < 1%).
- Ingestion latency (e.g., p95 < 5 seconds).
- Monitoring availability (e.g., ≥ 99.9%).
- Alert accuracy (true vs. false positives).
- Investigation time (e.g., MTTR improvement).

Key Takeaways

- Design observability to stay useful during partial outages.
- Control telemetry volume with sampling and limits.
- Use high-availability collectors and buffering.
- Alert on customer impact (SLO/SLI), not just noise.
- Regularly test failure and high-traffic scenarios.

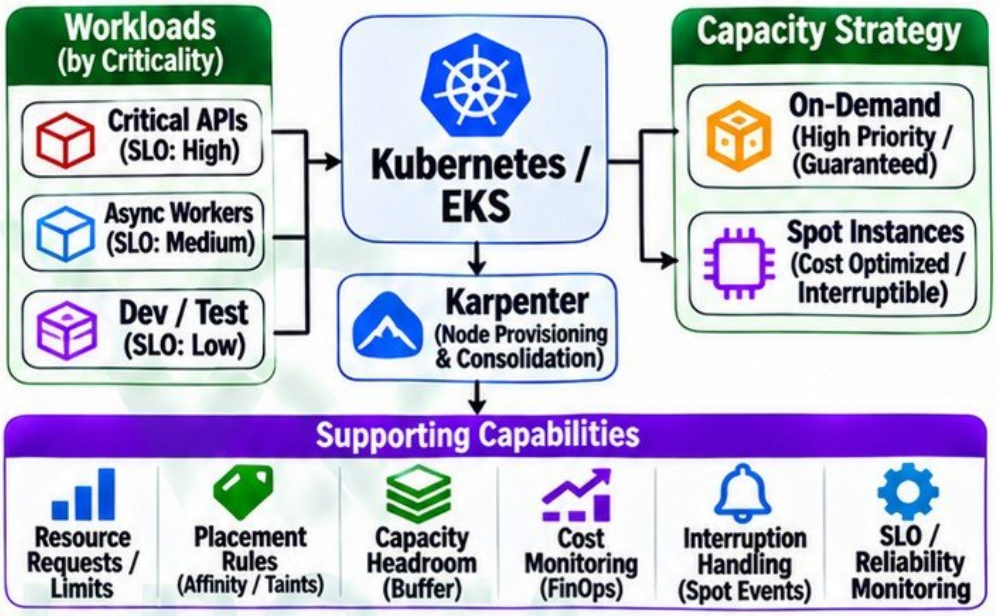
The Kubernetes Cost Reduction That Destroyed Reliability

Project 17 Build a FinOps-aware capacity management and workload placement system.

Project Objectives

- Compare on-demand and Spot capacity across workload criticality levels.
- Evaluate Karpenter consolidation, resource requests, limits and idle capacity.
- Build workload placement rules for critical APIs, async workers and development jobs.
- Simulate Spot interruption, capacity shortages and aggressive consolidation.

Architecture: FinOps-Aware Kubernetes Platform



Production Challenge

A cost-saving configuration removes the spare capacity required to survive an infrastructure failure, causing service degradation and SLO breaches.

Implementation Steps

1. Deploy workloads with defined criticality and SLOs.
2. Configure Karpenter with on-demand and Spot node pools.
3. Set resource requests, limits and consolidation policies.
4. Implement placement rules (critical, async and dev/test).
5. Simulate Spot interruption and node consolidation.
6. Measure cost, reliability and SLO impact, then optimize.

Evidence and Validation

- Monthly cost estimate (before vs. after).
- Workload interruption rate (e.g., Spot events).
- SLO impact (availability and latency).
- Capacity headroom (buffer during failure).
- Cost savings vs. reliability risk (trade-off analysis).

Key Takeaways

- Optimize cost without eliminating resiliency buffers.
- Use on-demand for critical workloads, Spot for fault-tolerant tasks.
- Tune Karpenter consolidation to avoid over-aggressive scaling.
- Maintain measurable SLOs and capacity headroom.
- Continuously review cost, utilization and failure impact.

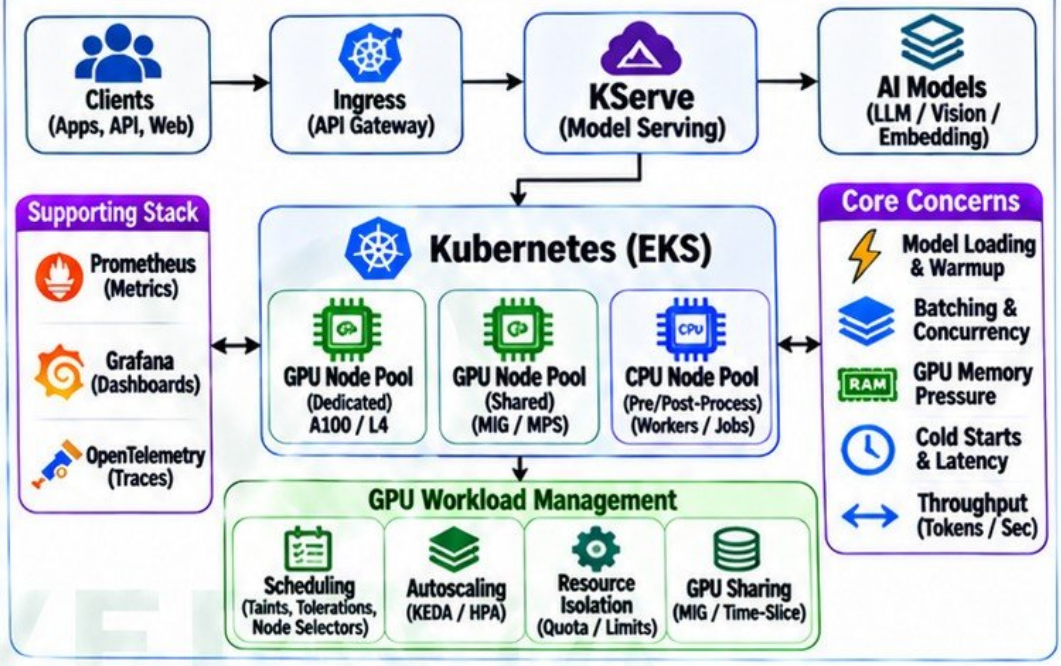
The AI Inference Platform That Wasted Expensive GPUs

Project 18 Build a production Kubernetes platform for GPU inference workloads.

Project Objectives

- Deploy model-serving workloads using KServe or a suitable inference stack.
- Schedule GPU workloads with proper node provisioning and resource isolation.
- Investigate model loading, batching, concurrency, cold starts and GPU memory pressure.
- Compare dedicated GPU allocation with supported GPU-sharing approaches.

AI Inference Platform Architecture



Production Challenge

GPUs appear heavily allocated, but actual inference throughput is low while request latency increases.

Implementation Steps

1. Deploy a model-serving platform (e.g., KServe) on EKS.
2. Provision GPU node pools (dedicated and shared/MIG).
3. Configure model loading, batching and concurrency settings.
4. Set up autoscaling and resource isolation (quotas, limits, taints).
5. Run load tests and compare dedicated vs. shared GPU allocation.
6. Measure throughput, latency and GPU utilization, then optimize.

Evidence and Validation

- GPU utilization (e.g., ≥ 70% steady).
- Throughput (e.g., tokens or inferences/sec).
- p95 and p99 latency (e.g., < target SLO).
- Queue time (e.g., < specified threshold).
- Cost per request (before vs. after optimization).

Key Takeaways

- High GPU allocation does not always mean high throughput.
- Tune model loading, batching and concurrency.
- Use the right GPU strategy (dedicated vs. sharing).
- Autoscale and isolate GPU workloads effectively.
- Measure both performance and cost continuously.

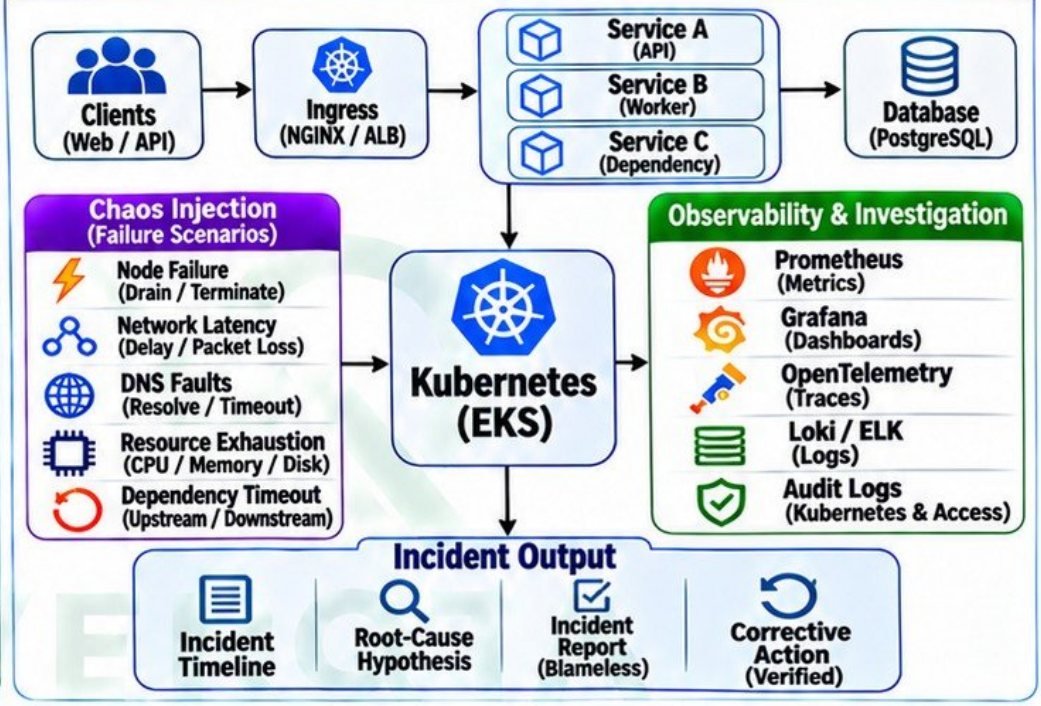
The Production Incident Nobody Could Reproduce

Project 19 Build a Kubernetes incident reconstruction and controlled chaos platform.

Project Objectives

- Create a production-like microservice system with hidden failure dependencies.
- Inject node failures, network latency, DNS faults, resource exhaustion and dependency timeouts.
- Correlate Kubernetes events, traces, metrics, audit records and application logs.
- Write a blameless incident report and validate corrective actions through repeated experiments.

Architecture: Incident Reconstruction Platform



Production Challenge

A five-minute outage disappears before engineers can inspect the failing pods.

Implementation Steps

1. Deploy a microservice system on Kubernetees (EKS).
2. Add hidden dependencies (e.g., database, cache, external API).
3. Set up observability (Prometheus, Grafana, OpenTelemetry, Loki/ELK).
4. Inject failures (node, network, DNS, resources, timeouts) using a chaos tool.
5. Correlate events, logs, metrics and traces to identify the root cause.
6. Write a blameless incident report and verify the fix with repeat tests.

Evidence and Validation

- Incident timeline (detection to recovery).
- Root-cause hypothesis (with supporting data).
- Telemetry correlation (events, logs, metrics, traces).
- Reproduction test (same failure, consistent result).
- Verified corrective action (no recurrence in retest).

Key Takeaways

- Build systems that are observable and testable.
- Use chaos testing to uncover hidden dependencies.
- Correlate multiple signals (events, logs, metrics, traces).
- Practice blameless incident reporting and learning.
- Validate fixes with repeatable experiments.

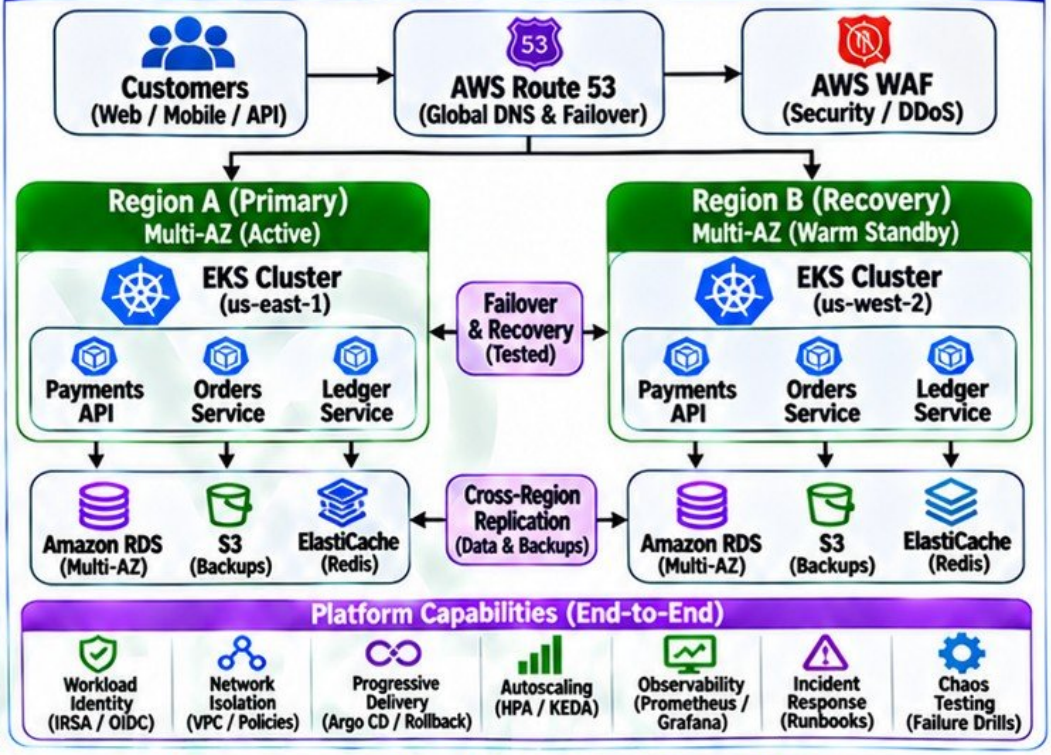
The Final Production Architecture Review: Can You Defend the Entire Platform?

Project 20 Design, deploy and defend a regulated multi-region transaction platform.

Project Objectives

- Build a complete Kubernetes platform for an international payment company.
- Implement multi-AZ availability and a tested regional recovery strategy.
- Secure identity, network isolation and software supply chain.
- Enable progressive delivery, automated rollback and GitOps governance.
- Ensure data protection, auditability, cost control, SLOs and incident response with chaos testing.

Architecture: Regulated Multi-Region Payment Platform



Production Challenge

During a peak transaction period, one AWS Region becomes unavailable, the recovery environment has limited capacity, and a newly deployed application version contains a defect.

Key Work Requirements

1. Design a multi-region, multi-AZ Kubernetes architecture.
2. Implement security, supply chain and network isolation controls.
3. Configure progressive delivery with automated rollback (GitOps).
4. Protect stateful data and ensure transaction integrity and auditability.
5. Enable autoscaling, cost controls and SLO-based monitoring.
6. Run incident response and chaos tests with clear operational ownership.

Evidence and Validation

- Regional failover and recovery test results.
- SLOs (availability, latency, error rate).
- Transaction integrity (no data loss or duplication).
- Deployment and rollback records (GitOps).
- Chaos test reports and operational ownership.

Key Takeaways

- Design for failure, not just high availability.
- Use multi-region, multi-AZ and tested recovery.
- Secure the platform end-to-end (identity, network, supply chain and data).
- Automate safe releases with fast, proven rollback.
- Validate resilience with chaos testing and clear ownership.